

Deterministic Tensor Governance via Interface-Led Architecture

A Contract-First Architecture for Governing LLM and Quantum Tensor Providers

Version: v0.1.0

Type: Position Paper / Architecture Note

Author: Takuya Sogawa

ORCID: <https://orcid.org/0009-0009-7499-2595>

License: CC BY 4.0

Note: This Japanese manuscript is a translation companion. The English manuscript is the canonical version.

Abstract

本稿は、大規模言語モデル（Large Language Model; LLM）および量子処理装置（Quantum Processing Unit; QPU）を含む異種テンソル Provider を対象とした、Contract-First な統治アーキテクチャを提案する。

本稿は、新たな低レイヤー量子 OS、量子コンパイラ、量子中間表現、またはハードウェア制御層を提案するものではない。むしろ、部分的に非決定論的な Provider を、決定論的な Interface、Contract、Invariant、Fail-Closed 境界、および監査可能な実行記録の内部に閉じ込めるための上位統治規律を定義する。

本稿の中心命題は、LLM の確率的サンプリングと QPU の測定・ノイズが物理的に同一現象であるということではない。両者の内部機構、物理的意味、評価方法は根本的に異なる。本稿が主張するのは、システム境界から見たとき、両者はいずれも「構造化された入力を受け取り、Provider 内部に局所化された非決定論性を経て、検証対象となる出力を返す計算資源」として、アーキテクチャ上類似したもの（architecturally analogous）として扱える、という点である。

本稿では、このような Provider を Interface-Led Architecture（ILA）によって統治するための上位設計原理を **Deterministic Tensor Governance** と呼ぶ。また、その実行境界を構成するガバナンス指向のアーキテクチャを **TensorKernel** と呼ぶ。

本稿の貢献は、LLM と QPU を統一的な物理モデルへ還元することではなく、非決定論的テンソル Provider をソフトウェア工学上の Contract、Invariant、ReplayLog、Policy Decision Point によって統治するための、引用可能な初期アーキテクチャ定義を提示することである。

Keywords

Deterministic Tensor Governance; Interface-Led Architecture; AIKernel; TensorKernel; LLM Governance; Quantum Governance; QPU; Contract-First Architecture; Fail-Closed; ReplayLog; Provider Model; Non-Deterministic Systems; Quantum Middleware; AI Governance

1. Introduction

現代の計算機科学は、完全に決定論的な命令列の実行だけでなく、確率的・非決定論的なテンソル計算資源のオーケストレーションを扱う段階に入っている。LLM は確率的サンプリングによる非決定性を持ち、QPU は測定、ノイズ、デバイス特性、実行条件に由来する非決定性を持つ。

これらは内部機構において同一ではない。LLM は学習済みパラメータとサンプリング過程に基づいてトークン列を生成する。一方、QPU は量子状態、量子ゲート、測定、および物理ノイズを含む実行過程を通じて、ビット列、測定分布、期待値、またはそれらに基づく推定値を返す。

しかし、エンタープライズシステムや自律実行環境から見れば、両者には共通する構造がある。すなわち、どちらも構造化された入力を受け取り、内部に局所化された非決定論性を経て、外部システムに影響を与え得る出力を返す Provider である。

このような Provider を安全に運用するためには、計算資源そのものの最適化だけでなく、その外側に配置される決定論的な制御プレーンが必要である。本稿では、AIKernel プロジェクトにおいて定義されてきた Interface-Led Architecture、Contract-First Development、Fail-Closed Governance、および ReplayLog の考え方を拡張し、LLM と QPU を含む非決定論的テンソル Provider を統治するためのメタアーキテクチャ原理を提案する。

2. Scope and Non-Claims

本稿の立場を明確にするため、最初に本稿が主張しないことを示す。

本稿は、以下を提案するものではない。

- 新しい量子 OS
- 新しい量子コンパイラ
- 新しい量子中間表現
- 新しい LLM アーキテクチャ
- LLM と QPU の物理的同一性
- QPU の測定過程を LLM のサンプリングで説明する統一物理理論
- 既存の QOS、QNodeOS、Pilot-Quantum、QIR、Qiskit、PennyLane、MLIRなどを置き換える実装

本稿が提案するのは、以下である。

- 非決定論的 Provider を Contract の内側に閉じ込める上位アーキテクチャ
- Provider の内部非決定性とシステム全体の決定論的制御を分離する設計原理
- LLM 統治モデルを QPU 統治へ転写するためのアーキテクチャ上の対応関係
- Fail-Closed、ReplayLog、Invariant、Policy Decision Point による監査可能な実行境界
- 既存 Quantum OS / Middleware / IR / SDK の外側に配置可能な governance layer の概念定義

したがって、本稿は下位レイヤーの計算制御ではなく、上位レイヤーの統治境界を扱う Position Paper である。

3. Related Work

TensorKernel および Interface-Led Architecture (ILA) は、既存の Quantum OS やハイブリッドミドルウェアを置き換えるものではない。むしろ、それらを Provider、Runtime、または Intermediate Representation として尊重し、その外側に Contract-First な governance layer を与える上位設計原理である。

3.1 Quantum OS and Middleware

QOS は、量子資源管理、ハードウェア制約、ノイズ、スケジューリング、マルチプログラミングなどを扱う Quantum Operating System として提案されている。QOS は量子計算資源を下位レイヤーで制御・最適化する重要な研究であり、本稿の TensorKernel はその外側に位置する統治レイヤーとして理解できる。

QNodeOS は、量子ネットワークノード上でアプリケーションを実行するための OS アーキテクチャとして提案されている。QNodeOS は量子ネットワーク上の実行環境を対象とする下位実行基盤であり、本稿の立場では Provider または Runtime として扱われる。

Pilot-Quantum は、Quantum-HPC 環境におけるリソース、ワークロード、タスク管理のためのミドルウェアである。これは、量子・古典ハイブリッド環境における実行管理を担う重要な基盤であり、本稿の governance layer はその実行判断や Provider 選択の外側に置かれる。

3.2 Quantum Frameworks and Intermediate Representations

Q#, Qiskit, PennyLane などの量子プログラミング環境は、Provider の実装基盤または利用者向けプログラミング層として扱うことができる。

QIR (Quantum Intermediate Representation) は、量子プログラミング言語やフレームワークと量子計算プラットフォームの間に位置する中間表現である。本稿では、QIR を TensorKernel の下位 Provider 境界における有力な Intermediate Representation として位置づける。

3.3 ML Compiler and Tensor Runtime

MLIR や XLA などのコンパイラ・ランタイム技術は、異種ハードウェア抽象化、最適化、コンパイル基盤として重要である。一方で、それらは主にコンパイル、変換、最適化、実行効率を対象とする。本稿の関心は、最適化された出力をさらに上位の Contract、Invariant、Policy、ReplayLog によって統治する点にある。

3.4 AIKernel and Interface-Led Architecture

AIKernel は、LLM を単なる API 呼び出しとしてではなく、Context、Contract、Provider、ReplayLog、Governance によって制御される実行対象として扱うプロジェクトである。筆者による Interface-Led Architecture (ILA) は、インタフェース、契約、責務境界を中心にソフトウェアを構築し、AI による実装生成を決定論的構造の内側に閉じ込める方法論として公開されている。

本稿は、この ILA の考え方を、LLM だけでなく QPU を含む非決定論的テンソル Provider へ拡張する。

4. Problem Statement

既存のハイブリッド計算アーキテクチャは、次の課題を抱えている。

4.1 Provider-Local Non-Determinism

LLM や QPU が持つ非決定性は、本来 Provider 内部に局所化されるべきである。しかし、Contract、Invariant、ReplayLog、Policy Decision Point が不足している場合、その非決定性はシステム全体へ波及する。

その結果、以下の問題が発生する。

- 出力がどの条件で許可されたかを説明できない
- 失敗時にどの境界で停止すべきかが不明確になる
- Provider の差し替えによってシステム全体の振る舞いが不安定化する
- 実行結果の検証可能性 (verifiability) が低下する
- 監査可能性 (auditability) が確保できない

4.2 Absence of Deterministic Control Boundaries

未知の複雑性、許容不可能なノイズ、低信頼な出力、または危険な操作候補に直面した際、システムを安全に停止させるためには、アーキテクチャレベルで厳格な Contract が必要である。

非決定性は排除できるとは限らない。むしろ、非決定性は Provider 内部に局所化され、外側の制御プレーンによって観測・評価・制限されなければならない。

本稿の中心的な問題意識は次のとおりである。

非決定論的 Provider を利用するシステムにおいて、非決定性をどのように Contract の内側へ閉じ込め、システム全体の決定論的統治を維持するか。

5. Non-Deterministic Tensor Provider Model

本稿では、LLM と QPU を **Non-Deterministic Tensor Provider** として抽象化する。

ここでいう「Tensor」は、狭義の数値テンソルだけを意味しない。構造化された高次元の入出力表現、トークン列、確率分布、測定分布、ビット列、期待値、回路記述、実行メタデータなどを含む、Provider 境界上で扱われる構造化データ表現を広く指す。

5.1 LLM Provider

LLM Provider は、概念的に次の構造を持つ。

```
Context / Prompt
↓
Probabilistic Sampling
↓
Token Sequence / Probability Distribution / Tool Candidate
```

LLM Provider の出力は、自然言語、構造化 JSON、ツール候補、コード片、推論軌跡などとして現れる。これらはそのまま信頼されるべきではなく、Contract、Schema、Policy、Risk Evaluation によって検証される必要がある。

5.2 QPU Provider

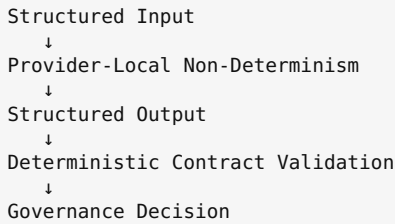
QPU Provider は、概念的に次の構造を持つ。

```
Quantum Circuit / QIR / Execution Request
↓
Quantum Execution / Measurement / Noise
↓
Bitstrings / Counts / Expectation Values / Measurement Statistics
```

QPU Provider の出力は、単一の決定論的値ではなく、測定結果、分布、統計的推定、誤差を含む形で返されることが多い。したがって、出力の意味は Post-Processing、Error Mitigation、Confidence Criteria、Contract Validation によって評価される。

5.3 Common Boundary Model

LLM と QPU は内部機構において同一ではない。しかし、アーキテクチャ境界上では次の共通構造を持つ。



この共通構造により、異なる物理的・計算的性質を持つ Provider であっても、外側の governance layer から統一的な Contract-First モデルで扱うことができる。

6. Interface-Led Architecture for Tensor Governance

Interface-Led Architecture (ILA) は、実装やハードウェアの特性ではなく、システム間の Contract を設計の起点とする。

Tensor Governance における ILA は、次の原則に基づく。

6.1 Contract-First

Provider を実装する前に、以下を定義する。

- 入力形式
- 出力形式
- 許容される不確実性
- 必須メタデータ
- 失敗条件
- Invariant
- 検証方法
- 監査ログ要件

Contract は、Provider の内部実装に依存しない。Provider は、Contract を満たす限り差し替え可能である。

6.2 Invariant Preservation

Invariant は、Provider の内部非決定性にかかわらず破ってはならない条件である。

例:

- 不正な出力を成功として扱わない
- 検証不能な出力を自動実行しない
- 危険な操作を確認なしに実行しない
- 実行判断を ReplayLog に残す
- Provider の失敗を安全な成功として隠蔽しない
- 未知のリスクを低リスクとして扱わない

6.3 Fail-Closed

Fail-Closed は、出力が Contract または Invariant を満たさない場合に、その境界で実行を停止する原則である。

Fail-Closed は、単なる例外処理ではない。それは、非決定論的 Provider をシステム全体へ直接接続しないための統治境界である。

6.4 Provider Replacement Boundary

Provider は Interface の背後に配置される。これにより、LLM、QPU、Simulator、Classical Tensor Runtime、Hybrid Runtime は、共通の governance layer から扱える。

```
ITensorProvider
├── LLMPProvider
├── QPUPProvider
├── QuantumSimulatorProvider
├── ClassicalTensorProvider
└── HybridRuntimeProvider
```

重要なのは、Provider の内部を統一することではなく、Provider 境界で観測・検証・記録・拒否できるようにすることである。

7. Deterministic Governance Skeleton

非決定論的 Provider を安全に運用するための TensorKernel は、次の Governance Skeleton によって構成される。

```
Request
↓
Policy Decision Point
↓
Contract Selection
↓
Provider Routing
↓
Provider Execution
↓
Result Verification
↓
Fail-Closed Gate
↓
ReplayLog
↓
Approved Output / Rejected Output
```

7.1 Policy Decision Point

Policy Decision Point (PDP) は、実行前にタスクの性質、複雑性、リスク、必要な信頼水準、利用可能な Provider を評価する。

PDP は次を判断する。

- どの Provider にルーティングするか
- 実行前に確認が必要か
- どの Contract を適用するか
- 失敗時の挙動をどうするか
- 出力に必要な証跡は何か

7.2 Contract Registry

Contract Registry は、Provider ごとの契約を管理する。

例:

```
Contract
├── Input Schema
├── Output Schema
├── Invariants
├── Risk Policy
├── Verification Rule
├── Replay Requirement
└── Fail-Closed Behavior
```

7.3 Provider Adapter

Provider Adapter は、下位 Provider の固有 API、SDK、Runtime、IR を TensorKernel の共通 Interface へ写像する。

QPU の場合、Adapter は QIR、Qiskit、PennyLane、または特定の Quantum Runtime と接続できる。LLM の場合、Adapter は OpenAI-compatible API、local model runtime、tool-calling API などと接続できる。

7.4 Result Verifier

Result Verifier は、Provider 出力が Contract を満たすかを検証する。

LLM では、JSON schema validity、tool call validity、semantic alignment、risk score、policy compliance などを検証できる。

QPU では、shot count、measurement distribution、confidence interval、error threshold、expected observable consistency、backend metadata などを検証できる。

7.5 ReplayLog

ReplayLog は、すべてのガバナンス判断と Provider 入出力を不変のログとして記録する。

ReplayLog は、Provider 内部の非決定性そのものを消すものではない。しかし、少なくとも制御プレーン上の判断軌跡を再構成可能にする。

記録対象は次を含む。

- trace id
- request metadata
- selected contract
- selected provider
- provider input
- provider output summary
- verification result
- risk score
- decision
- fail-closed reason
- timestamp
- versioned policy id

8. LLM-to-QPU Governance Transfer

本稿の独自性は、AIKernel において形式仕様化・設計検証されてきた LLM 統治モデルを、QPU という異なる物理的計算資源の統治へ転写する点にある。

ここでいう Transfer は、物理的同一性の主張ではない。あくまで、アーキテクチャ境界上の統治パターンの転写である。

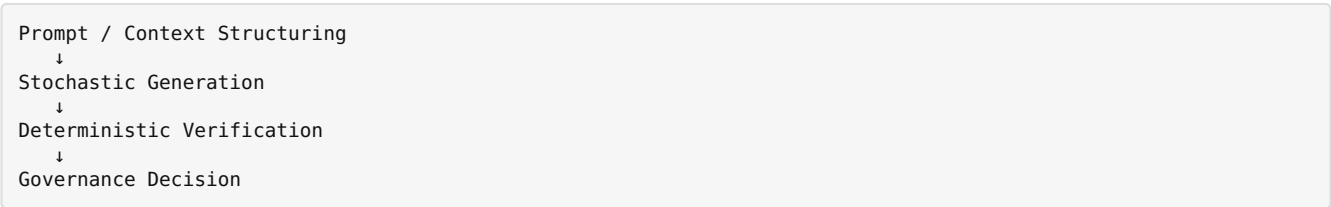
8.1 Analogical Mapping

LLM Governance	QPU Governance
Prompt / Context	Quantum Circuit / QIR / Execution Request
Sampling	Measurement / Device Noise / Shot Execution
Token Sequence	Bitstrings / Counts / Expectation Values
Tool Candidate	Quantum Job / Backend Execution Result
Schema Validation	Measurement Result Validation
Semantic Alignment	Observable / Circuit Intent Consistency
Risk Evaluation	Backend / Noise / Cost / Confidence Evaluation
ReplayLog	Experiment / Execution / Governance Log
Fail-Closed Gate	Reject / Re-run / Require More Shots / Abort

この対応関係により、LLM 統治で得られた設計原理を、QPU 統治へアーキテクチャ上類似した形で移植できる。

8.2 Structured Governance Pipeline

LLM における統治パイプラインは、概念的に次のように表せる。



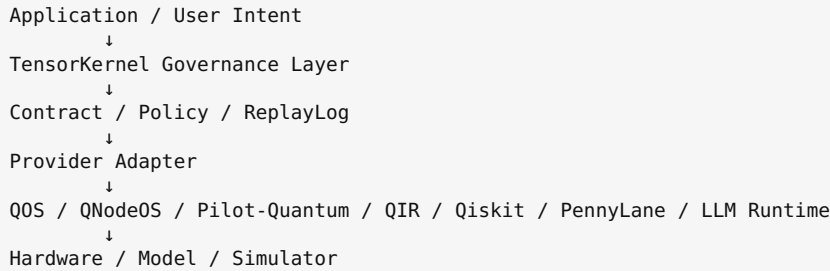
これを QPU に転写すると、次のようになる。



この対応関係は、LLM と QPU の内部メカニズムが同一であることを意味しない。それは、両者を非決定論的 Provider として外側から統治するための共通構造が存在することを意味する。

9. Architectural Layering

TensorKernel は、既存の Quantum OS、Quantum Middleware、Quantum IR、LLM Runtime、ML Compiler を置き換えない。むしろ、それらの外側に配置される。



この構造では、下位レイヤーは実行能力を提供し、TensorKernel は実行の許可、拒否、監査、Contract 検証を担う。

10. Security and Auditability

非決定論的 Provider を扱うシステムでは、Provider 出力を信頼境界の外側に置く必要がある。

TensorKernel は、次の原則を採用する。

1. Provider output is untrusted by default.
2. Execution requires explicit governance approval.
3. Unknown risk increases the risk score.
4. Invalid or unverifiable output must not be silently accepted.
5. Dangerous execution requires confirmation or rejection.
6. Provider routing must be policy-controlled.
7. ReplayLog must preserve the evidence for governance decisions.
8. Fail-Closed behavior must be the default.

これらの原則により、LLM や QPU の出力がそのまま業務システムや物理的リソースに影響することを防ぐ。

11. Limitations

本稿にはいくつかの限界がある。

11.1 Conceptual Stage

本稿は Position Paper であり、TensorKernel の完全な実装や評価結果を提示するものではない。したがって、本稿の主張はアーキテクチャ原理の提示に留まる。

11.2 Analogy Limitation

LLM と QPU は内部機構において根本的に異なる。本稿の対応関係は、物理的同型性ではなく、システム境界上のアーキテクチャ類似性に基づく。

11.3 Verification Limitation

Contract 検証は万能ではない。特に QPU では、統計的信頼区間、ショット数、ノイズモデル、Backend metadata などに依存するため、検証は確率的・統計的な性質を持つ。

11.4 Governance Overhead

PDP、Contract Registry、Result Verifier、ReplayLog は実行コストを増加させる可能性がある。したがって、低リスク・低コストのタスクでは軽量の governance profile を用い、高リスクな実行では厳格な profile を用いる必要がある。

11.5 Terminology Risk

TensorKernel という名称は、量子 OS や GPU kernel と誤解される可能性がある。本稿における TensorKernel は、低レイヤー実行 kernel ではなく、Contract-First な governance layer を指す。

12. Future Work

今後の作業として、次を予定する。

1. ITensorProvider、ITensorContract、ITensorReplayLog などの最小 Interface 定義
2. LLM Provider を対象とした AIKernel.NET 上での governance prototype
3. Quantum Simulator Provider を対象とした疑似 QPU governance 実験
4. QIR または Qiskit を入力とする Provider Adapter の設計
5. ReplayLog を用いた Provider 実行の監査可能性評価
6. Risk score と confidence criteria の初期モデル化
7. 多値論理、p 進数空間、確率的型システムなどの代数的拡張の検討

13. Conclusion

本稿では、LLM と QPU を、内部機構や物理的意味の違いを保持したまま、アーキテクチャ境界上では Non-Deterministic Tensor Provider として類似的に扱い、Interface-Led Architecture に基づいて統治する上位設計原理を提案した。

本稿の目的は、既存の Quantum OS、Quantum Compiler、Quantum Middleware、Intermediate Representation、LLM Runtime を置き換えることではない。それらを Provider、Runtime、または Intermediate Representation として尊重し、その外側に Contract-First な governance layer を配置することで、非決定性を許容しながらも、システム全体の決定論的制御、監査可能性、Fail-Closed 安全性を維持することにある。

Deterministic Tensor Governance は、LLM と QPU を単一の物理理論に還元するものではない。それは、非決定論的計算資源をソフトウェア工学の Contract、Invariant、Interface、ReplayLog、Policy Decision Point によって統治するためのアーキテクチャ原理である。

References

1. Giortamis, Emmanouil, Francisco Romão, Nathaniel Tornow, and Pramod Bhatotia. “QOS: Quantum Operating System.” 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25), USENIX Association, 2025, pp. 429–447.
2. Delle Donne, C., M. Iuliano, B. van der Vecht, G. M. Ferreira, H. Jirovská, T. J. W. van der Steenhoven, A. Dahlberg, et al. “An Operating System for Executing Applications on Quantum Network Nodes.” *Nature*, vol. 639, no. 8054, 2025, pp. 321–328. DOI: 10.1038/s41586-025-08704-w.
3. Mantha, Pradeep, Florian J. Kiwit, Nishant Saurabh, Shantenu Jha, and Andre Luckow. “Pilot-Quantum: A Quantum-HPC Middleware for Resource, Workload and Task Management.” arXiv:2412.18519, 2024. DOI: 10.48550/arXiv.2412.18519.
4. Microsoft. “Q# Quantum Programming Language.” Microsoft Learn.
5. IBM. “Qiskit SDK.” IBM Quantum Documentation.
6. Bergholm, Ville, Josh Izaac, Maria Schuld, Christian Gogolin, M. Sohaib Alam, Shahnawaz Ahmed, Juan Miguel Arrazola, et al. “PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations.” arXiv:1811.04968, 2018.
7. Microsoft. “Quantum Intermediate Representation.” Microsoft Learn.
8. Lattner, Chris, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, et al. “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation.” 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 2021.
9. Sogawa, Takuya. “AIKernel Trajectory Governance Model: A Kernel-Level Framework for Convergent Decision Control over Stochastic Language Model Inference.” AIKernel.NET Phase-1 Specification, 2026.
10. Sogawa, Takuya. “Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model.” Zenodo, 2026. DOI: 10.5281/zenodo.20290614.